

Cron at the center of your pipelines

Your Cron deserves a second version: from orphan script to supervised service

The context

In my previous post I talked about why Cron alone is a risk for critical tasks. Now the practical part: how do you migrate? The trick is that you don't need to rewrite your business logic. You just need to wrap it.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=10)
def job(data):
    return {"done": run(data["command"])}

pipe = Pipeline(pipeline_name="cron_pro", tracking_db="cron.db")
pipe.set_steps([job])

# Keep the schedule; gain checkpoints + audit.
```

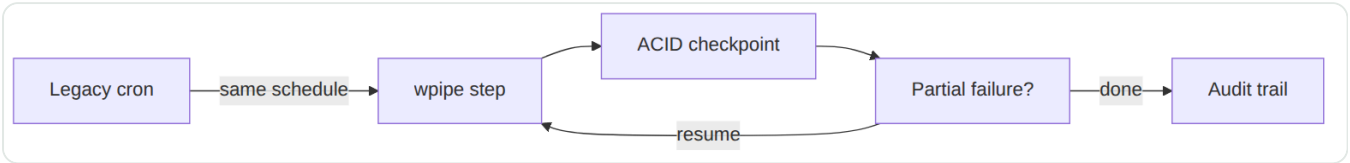
At a glance

Capability	Cron	Cron + wpipe
Retries	✗ None	✓ Scheduled
Checkpoints	✗ Restarts from zero	✓ Resumes the step
Logs	Plain text	✓ Searchable SQL Tracker
Alerts	Nothing	✓ Configurable thresholds

Capability	Cron	Cron + wpipe
Dashboard	No	✅ Realtime on :5000

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads
A community of efficiency-first developers.

<50MB RAM
Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL
Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore Cron workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.